Embedding и векторные базы данных

План

- Embedding
- Векторные базы данных
- Поиск в векторных БД
- Чанкинг, метаданные, reranking

Embeding

Эмбеддинг — это плотный вектор (массив чисел с плавающей запятой) фиксированной длины, который представляет семантическое (смысловое) значение текста, изображения или аудио.

Векторные базы данных: зачем они и как работают?

Традиционные базы данных (SQL, Elasticsearch) отлично ищут по точному совпадению (keyword search), но плохо понимают смысл. Если пользователь ищет "средство от головной боли", а в базе написано "обезболивающее", SQL-запрос вернет пустоту.

Векторная БД решает эту проблему. Она хранит векторы и использует алгоритмы ANN (Approximate Nearest Neighbor), такие как HNSW (Hierarchical Navigable Small World), чтобы молниеносно находить ближайшие векторы в многомерном пространстве, не перебирая всю базу целиком

Векторные базы данных: зачем они и как работают?

Основные метрики расстояния:

- Cosine Similarity: Измеряет угол между векторами. Идеально для текста, нормализованного по длине.
- Euclidean Distance (L2): Прямое расстояние между точками. Чувствительно к длине вектора.
- Dot Product: Скалярное произведение (часто используется, если векторы уже нормализованы, так как математически равно косинусному сходству).

Создание и поиск по эмбеддингам с Chroma

Chroma — это векторная БД с открытым исходным кодом, разработанная специально для работы с LLM. Её главное преимущество: она может работать полностью в памяти (in-memory) или в локальном файле, что идеально для прототипирования.

Qdrant — это высокопроизводительная векторная БД на Rust. В отличие от Chroma, она предназначена для production-среды, поддерживает сложную фильтрацию по метаданным (payload) и масштабируется.

Полезные практики: чанкинг, метаданные, reranking

При работе с большими документами (книги, статьи) мы не можем скормить их в модель целиком из-за лимита токенов и "потери внимания" (Lost in the middle phenomenon).

Чанкинг (Chunking): Разбиение текста на небольшие фрагменты (чанки).

Правило: Чанк должен быть достаточно маленьким, чтобы содержать одну мысль, но достаточно большим, чтобы иметь контекст. Типичный размер: 300–500 токенов.

Полезные практики: чанкинг, метаданные, reranking

Метаданные (Metadata): К каждому чанку нужно прикреплять контекст: {"file_name": "lecture_1.pdf", "page": 12, "author": "Ivanov"}. Это позволяет LLM в ответе давать ссылки на источник, а векторной БД — фильтровать поиск.

Reranking (Переранжирование): Паттерн "Retrieve and Rerank".

- Этап 1 (Retrieval): Векторная БД быстро находит топ-50 наиболее похожих чанков (используя легкие Bi-encoder модели). Это быстро, но не всегда точно.
- Этап 2 (Reranking): Специальная модель (Cross-encoder) попарно сравнивает запрос и каждый из 50 чанков, выдавая точную оценку релевантности (score от 0 до 1). Мы берем топ-5 лучших и отдаем их в LLM. Это медленнее, но намного точнее.

Расширенные стратегии чанкинга и реранкинга

Продвинутый чанкинг

Простое разбиение по символам (text.split(',')) рвет предложения и теряет смысл.

- **Recursive Character Text Splitter** (стандарт LangChain): Пытается разбить по абзацам (\n\n), если не получается — по предложениям (.), затем по словам ( ).
- **Overlap** (Перекрытие): Обязательно добавляйте 10-20% перекрытия между чанками. Если предложение обрывается на конце чанка А, его начало будет в чанке Б, сохраняя семантический контекст для эмбеддинга.
- **Semantic Chunking**: Разбиение текста не по символам, а там, где резко меняется косинусное сходство между соседними предложениями (граница смысловых блоков).

Расширенные стратегии чанкинга и реранкинга

Тип данных / Задача	Рекомендуемая стратегия	Инструменты / Библиотеки
Обычные тексты, статьи, книги	Recursive Character + Overlap (10-15%)	LangChain RecursiveCharacterTextSplitter
PDF с таблицами, колонками, схемами	Document-Aware (Vision-based parsing)	LlamaParse, Unstructured.io, Marker
Исходный код (Python, JS, Java)	AST-based Chunking (по функциям/классам)	LangChain Language" splitter, Tree-sitter
Сложные технические мануалы	Semantic Chunking	LlamaIndex SemanticSplitterNodeParser
Высокие требования к точности ответа	Parent-Document Retriever (Small-to-Big)	LangChain ParentDocumentRetriever
Пользователи формулируют запросы неточно	LLM-обогащение (Hypothetical Questions)	Кастомный пайплайн индексации

Продвинутый реранкинг

Использование Cross-Encoder моделей (например, cross-encoder/mmarco-mMiniLM-L-6-v2 или BAAI/bge-reranker-large)

RAG Architecture

